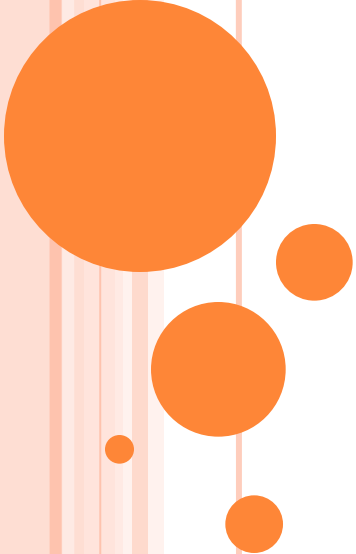




SEMANTICA OPERACIONAL

Formas de comunicación entre las rutinas - Parámetros

Rutinas

- En la **bibliografía** y en distintos **lenguajes** pueden recibir distintos nombres:
 - **subprogramas, subrutinas, funciones, procedimientos**
- Todos estos conceptos representan una **unidad de abstracción** que permiten:
 - **encapsular** un conjunto de **sentencias**
 - **definir** nuevas **operaciones**
 - **invocar** mediante un **call/return** con una **llamada explícita** por su **nombre** y luego **retornar a algún punto de la ejecución**

¿Qué pasaría si no existieran rutinas, que perderíamos?
modularidad, reutilización, claridad en el diseño

Rutinas

- Formas de **Subprogramas**
 - **Procedimientos:**
 - **no devuelven un valor directamente**
 - **Funciones:**
 - **si devuelven un valor como resultado de la invocación**

**Son semánticamente distintos
veamos sus características**

Rutinas

■ Tipo:

■ Procedimientos

- Son **construcciones que dan nombre a un conjunto de sentencias y declaraciones asociadas**, que se usarán para **resolver un subproblema** dado.
- Brindará una **solución de código más corta, comprensible y fácilmente modificable**.
- Permiten al **programador definir y crear nuevas acciones**, que el **programador invocará**
- **Pueden no recibir ni devolver ningún valor.**
- Al ser **invocado**, el **control se transfiere al procedimiento** y, al **finalizar**, **retorna al punto de invocación, continuando la ejecución del programa.**
- Los **resultados** los produce en:
 - **variables no locales**
 - **Parámetros** que **cambian su valor.**

Rutinas

■ Tipo:

$x := f(3) + 2;$

■ Funciones

- Son subprogramas que **permiten ejecutar** un **conjunto** de **sentencias** y **devuelven** un **valor** al **punto** donde fue **invocada**.
- Permiten al **programador** **definir nuevas operaciones**
- Se las **invoca dentro** de **expresiones**, y el **valor** que **retornan** **reemplaza a la llamada** en dicha **expresión**.
- Son **similares** a las **funciones matemáticas**, ya que **reciben datos** y **producen** un **resultado**.
- en **ciertos lenguajes** de **programación** pueden **producir efectos colaterales** (según el lenguaje)
- **Toda función debe retornar un valor.**

Rutinas/Subprogramas

Conclusiones:

- **Abstracción de Subprogramas**
 - Un **subprograma** es la **implementación** de una **acción abstracta**
 - Su **invocación** representa **el uso de dicha abstracción**.
 - Con **una** sola **definición** se pueden crear **muchas activaciones** durante la ejecución.
 - **Definir un subprograma** equivale a **incorporar una nueva operación al lenguaje**.

- **Ocultamiento de implementación (encapsulamiento)**
 - **Separa el qué hace del cómo lo hace**
 - El **programador no necesita conocer la implementación**
 - La **implementación** permanece **oculta**

Rutinas/Subprogramas

■ Problema central

Dividimos un programa en múltiples rutinas pero surge un problema:

¿Cómo intercambian información entre sí?

- **¿Quién controla** esos datos
- **¿Dónde se almacenan** esos datos?
- **¿Cuándo se modifican** esos datos?

Formas de conectar y "compartir datos" entre diferentes Unidades de Programa:

Resumen de comunicación:

1. **Variables locales** (no hay comunicación entre rutinas)
2. **Variables no locales** (lo que está en el entorno)
 - 1) **Ambiente no local implícito** (menos claro)
 - Utiliza **regla de Alcance dinámico** (quién la llamó)
 - Utiliza **regla de Alcance estático** (dónde está contenido)
 - 2) **Ambiente común explícito** (interviene el programador)
 - Se definen **áreas comunes** de código
3. **Parámetros** (comunicación explícita, no depende del Ambiente no local)
 - la mejor forma de compartir, más clara y recomendada

Formas de conectar y "compartir datos" entre diferentes Unidades de Programa:

1) A través del acceso al ambiente no local

compartir variables que son **de otra unidad** con un **acceso no local** (es **menos claro**, puede llevar a errores)

■ Ambiente no local implícito

Es automático de acuerdo a la regla de alcance del lenguaje, no por programador

- ✓ **regla de alcance dinámico** (quién me llamó y busco el identificador)
- ✓ **regla de alcance estático** (dónde está contenido y busco el identificador)

Formas de conectar y "compartir datos" entre diferentes Unidades de Programa:

Procedure Main;

var x,z,n: integer;

Procedure A1()

var m: integer;

Begin

m:=3; x:= x+m+1; z:=z+1+n;

end;

Procedure A2()

var x, z: integer;

Procedure A3();

var z, n: integer;

begin

n:=3; z:= x + n; **A1();**

end;

begin

x:= 1; z:= x +n; **A3();**

end;

begin

x:=2; z:=1; n:=4; **A2();**

end.

- **Analizar a través del acceso al Ambiente no local – Implícito**

¿Cuál es el Resultado?

todos usan x z n

Main llama A2(),

A2() llama a A3(),

A3() llama a A1(),

A1() usa x,z,n,m

El resultado dependerá de comparar si usa alcance dinámico o alcance estático

Formas de conectar y "compartir datos" entre diferentes Unidades de Programa:

1) A través del acceso al ambiente no local

- Ambiente común explícito

- Lo **define** el **programador**, debe **especificar** que **variables comparte**
- Se **definen áreas de memoria compartida** donde se **almacenan variables accesibles por múltiples rutinas**
- **Cada lenguaje** tiene **su forma**

- Ejemplos :

- **ADA:** uso de **paquetes** con *cláusula* **PACKAGE** (para **Tipos Abstractos de Datos - TAD**)
- **PL/1:** **variables externas** con *cláusula* **DECLARE**,
- **FORTRAN:** **área común** con *cláusula* **COMMON**

Formas de conectar y "compartir datos" entre diferentes Unidades de Programa:

2) A través de Pasaje de Parámetros

- **comunicación** es **explícita** (que se envía y que se recibe)
- **mejor** forma de **compartir**, más **clara** y **recomendada**

– **Parámetro Real (Argumento):**

- Es el **valor, variable o expresión** que **se pasa en la invocación a una rutina**
- **Se especifica en la llamada**

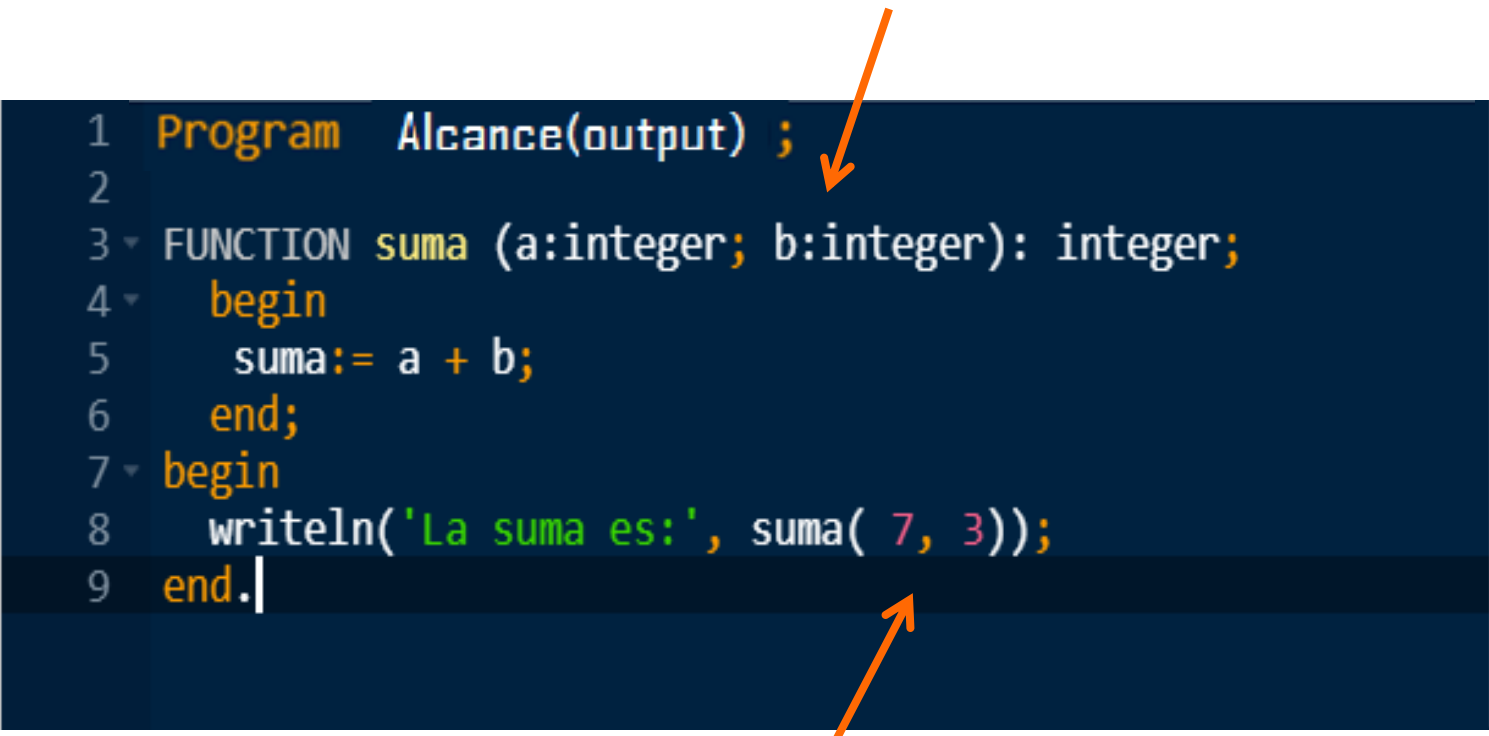
– **Parámetro Formal (Parámetro):**

- **Recibe el valor del parámetro real**
- Es la **variable *definida*** en la **declaración de la rutina**

Parámetros

Parámetros Formales (PARÁMETROS) → en declaración

```
1 Program Alcance(output) ;  
2  
3 FUNCTION suma (a:integer; b:integer): integer;  
4 begin  
5     suma:= a + b;  
6 end;  
7 begin  
8     writeln('La suma es:', suma( 7, 3));  
9 end.
```



The diagram illustrates the distinction between formal parameters and arguments in Pascal. An orange arrow points from the text 'Parámetros Formales (PARÁMETROS) → en declaración' to the parameter declarations '(a:integer; b:integer): integer;' in line 3 of the code. Another orange arrow points from the text 'Parámetros Reales (ARGUMENTOS) → en invocación' to the arguments '(7, 3)' in line 8 of the code.

Parámetros Reales (ARGUMENTOS) → en invocación

Formas de conectar y "compartir datos" entre diferentes Unidades de Programa:

Ventajas del uso de parámetros

El uso de parámetros es preferible al acceso al ambiente no local porque:

- **Hace explícita la comunicación entre rutinas**
- **Reduce la dependencia de variables no locales**
- **Mejora el control sobre los datos compartidos**

El acceso intensivo al ambiente no local puede:

- **Exponer variables donde no es necesario**
- **Reducir la claridad del programa**
- **Generar errores difíciles de detectar**

Ventajas del Pasaje de Parámetros

- **Flexibilidad**, permite **transferir más datos y de distinto tipo en cada llamada**.
- **Abstracción**, permite **compartir en forma más abstracta**. Sólo especificamos el **nombre de los argumentos/parámetros** y el **tipo** de cada cosa compartida.
- **protección**: sólo acceso a datos necesarios. En cambio el **acceso al ambiente no local** decrementa la **seguridad** ya que las **variables terminan siendo visibles** aun **donde no es necesario** o donde no deberían.

Ventajas del Pasaje de Parámetros

Legibilidad:

- **Más claro y entendible el código**
- **Evita accesos al ambiente no local**
- **Evita accesos repetidos a variables externas**
- **Sabemos qué se comparte,** para qué se usan y su tipo

Modificabilidad:

- **Facilita encontrar errores**
- Los **cambios** se **aplican** en **un solo lugar**
- **Facilita la depuración**

una buena cualidad de un programador es
minimizar el acceso a datos no locales

Parámetros

¿Los parámetros formales son variables locales?

- ***Sí, son variables locales a la rutina.***

¿Qué puede ser un parámetro real?

- ***un valor, variable, expresión, etc.,***
- ***cada lenguaje tiene sus reglas***

Vinculación de los Parámetros

La vinculación entre parámetros reales y formales comprende:

• Evaluación:

- Se **evalúan** los **parámetros reales** antes de la **invocación** (en general)
- Se **verifica** que **todo** esté **bien** antes de **transferir** el **control** a la **unidad llamada**

• Ligadura:

- Se **establece** la **correspondencia** entre **reales** y **formales**.
- **Puede ser de 2 tipos**
 - Por posición: Van en el mismo orden que la lista de parámetros.
 - Por Nombre o palabra clave: pueden estar **colocados en distinto orden** en la lista de parámetros.

Parámetros – Ligadura por nombre

- Evaluación de los parámetros reales y ligadura con los parámetros formales

Ejemplo por nombre (Python light)

Invocación: **P(y => 4; x => z);**

Declaración: **Procedure P (x: IN integer, y: IN float)**
P(z,4) x recibe z y recibe 4

Desventajas:

- ✓ Es necesario **conocer** los **nombres** de los **parámetros formales**
- ✓ **Puede generar errores** en la **asignación**

Parámetros

Tipos de Ligadura y valores por defecto según el lenguaje

- Cada lenguaje tiene sus reglas sintácticas y semánticas para el **pasaje de parámetros**. No ofrecen las mismas formas de **ligadura**, y esto **impacta** en la **claridad** y **flexibilidad** de las llamadas.
- En **ciertos lenguajes**, los **parámetros formales** pueden tener **valores por defecto**, por lo que **no** es **necesario especificarlos** todos en la **invocación**
- Los **valores por defecto** **reducen** la **cantidad** de **argumentos necesarios**, pero **no cambian** el **mecanismo** de **ligadura** **utilizado**

Lenguaje	Por posición	Por nombre	Combinación	Valores por defecto
C	✓	X	X	X
Java	✓	X	X	X
JavaScript	✓	X	X	✓
Python	✓	✓	✓	✓
Ada	✓	✓	✓	✓
C++	✓	X	X	✓

Parámetros

Ligadura y valores por defecto según el lenguaje

Ejemplo por defecto en Javascript:

```
function saludar(nombre = 'Miguel Angel') {  
  console.log('Hola ' + nombre);  
}
```

← **Parámetro predeterminado**

← **Salida "Hola Miguel Ángel"**

Esta función recibe un parámetro llamado "nombre", con un valor predeterminado. Este valor se asignará en caso que al invocar a la función no le pasemos nada.

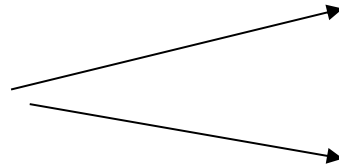
```
saludar();
```

← **Invocación sin parámetros**

Eso produciría la salida por consola "Hola Miguel Angel".

Parámetros

**Tipos de
Parámetros
(según
Información
Trasmitida)**



1) Datos

2) Subprogramas

- **Cada uno** tiene **distintos modos de uso**
- **Dependerá** de cada **lenguaje**

Parámetros

1) Datos pasados como Parámetros

Clasificación según cómo se comunica la información entre rutina llamadora y llamada :

- **Modo IN:**

- El **parámetro formal recibe** el **valor** desde el **parámetro real**.
- La **conexión es al inicio, se copia y se corta la vinculación**

- **Modo OUT:**

- **devuelve el parámetro formal al parámetro real** cuando termina la ejecución de la rutina.
- **La conexión es al final**

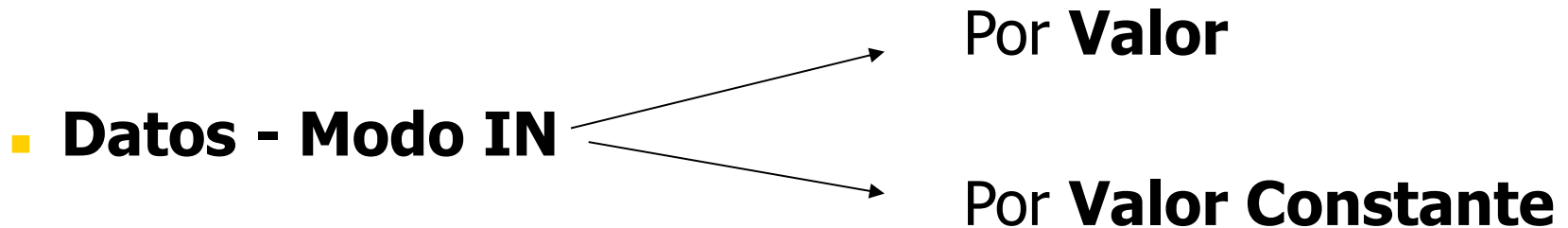
- **Modo IN/OUT:**

- El **parámetro formal recibe** el dato del **parámetro real**
- **Puede modificarse**
- El **parámetro formal envía** el dato al **parámetro real**
- **La conexión es al inicio y al final**

Datos

Parámetros

Diferentes formas de transmitir los parámetros



En ambos casos, el parámetro se utiliza solo como entrada y la conexión es al inicio

Parámetros

• Datos - Modo IN - por Valor:

- El valor del parámetro real se usa para inicializar el correspondiente parámetro formal al invocar la unidad.
- El dato real transferido se copia
- El parámetro formal pasa a ser una variable local de la unidad llamada (se crea otra variable)
- la **conexión es al inicio** al pasar el valor y **se corta la vinculación**.
- Es el mecanismo por **default** y el **más usado**

Desventajas:

- **consume tiempo** para hacer la **copia de cada parámetro**
- **consume almacenamiento** para **duplicar** cada **dato** (especialmente en *grandes estructuras o volúmenes*)

Ventajas:

- **Seguridad, protege los datos** de la unidad llamadora, el **parámetro real no se modifica**.
- **No hay efectos negativos o colaterales**

Parámetros

Datos - Modo IN - por Valor:

- ✓ **Analicemos** el comportamiento del **pasaje de parámetros por valor**, el **manejo de la pila (stack)**, y del **registro de activación**
- ✓ **Observemos** qué ocurre con el **parámetro real** y el **parámetro formal** durante la **ejecución**

Parámetros

Datos – Modos IN– Por Valor (Ejemplo: PASCAL)

Program main

```
var i:integer;
```

```
Procedure P(a:integer)
```

```
var x,y: integer;
```

```
Begin
```

```
  a=a+3;
```

```
  x=a+1;
```

```
  y=x+1;
```

```
end;
```

```
Begin
```

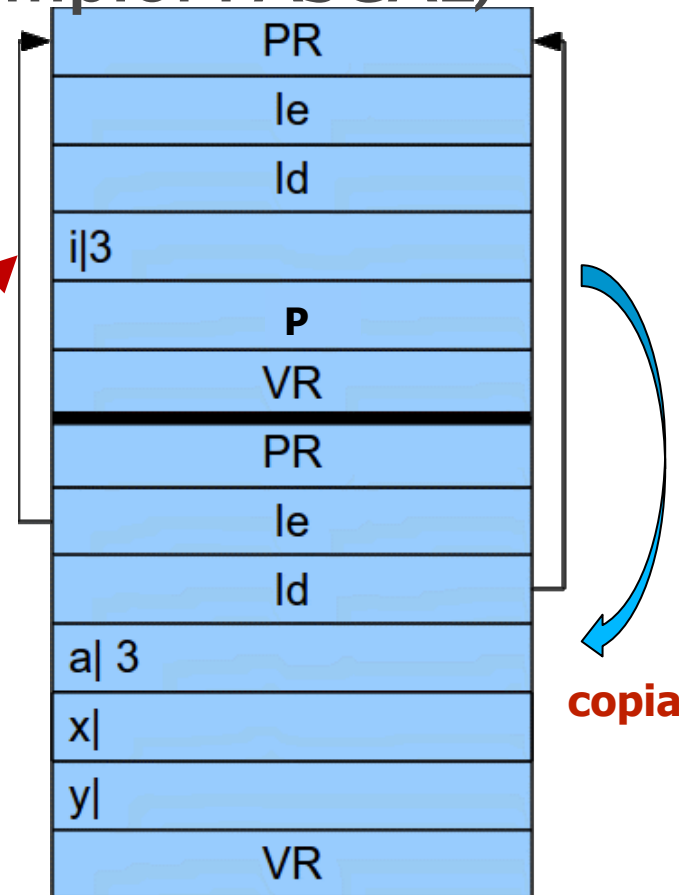
```
  i=3;
```

```
  P(i);
```

```
  Print(i);
```

```
End.
```

**Imprime
i (3)**



1. Se realiza la llamada a la rutina con un parámetro real
2. Al invocar se copia el valor del parámetro real en el parámetro formal
3. Durante la ejecución se trabaja con una copia independiente
4. Parámetro real no se modifica

Parámetros

Datos – Modos IN– Por Valor
Program main

```
var i:integer;
```

```
Procedure P(a:integer)
```

```
var x,y: integer;
```

```
Begin
```

```
  a=a+3;
```

```
  x=a+1;
```

```
  y=x+1;
```

```
end;
```

```
Begin
```

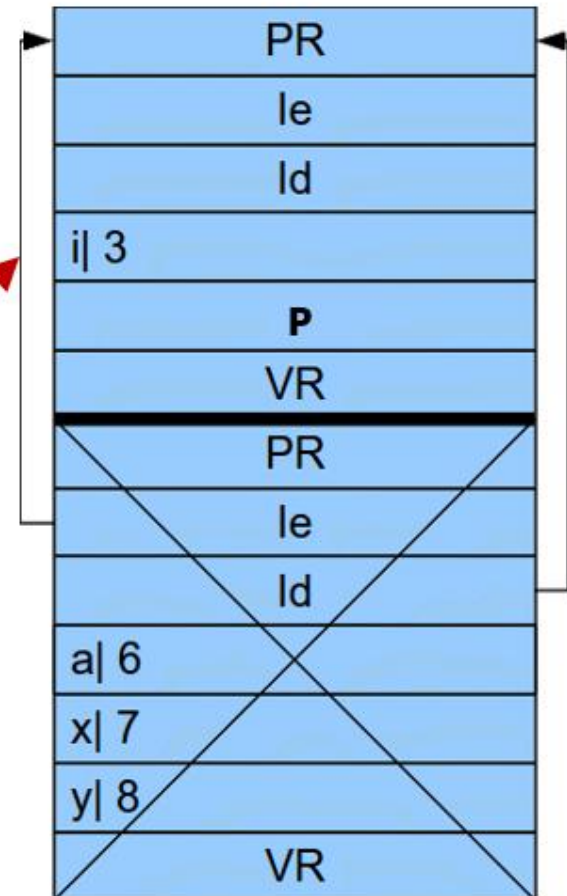
```
  i=3;
```

```
  P(i);
```

```
  Print(i);
```

```
End.
```

RESULTADO FINAL



Imprime
x=3

1. Se realiza la llamada a la rutina con un parámetro real
2. Al invocar se copia el valor del parámetro real en el parámetro formal
3. Durante la ejecución se trabaja con una copia independiente
4. Parámetro real no se modifica

Parámetros

Datos - Modo IN - Por valor constante:

- No todos los lenguajes permiten este tipo de pasaje
- Se **envía un valor**, pero:
 - la **rutina receptora no puede modificarlo**
 - El **parámetro** mantiene un **valor fijo**.
 - Son **Reglas de la semántica estática del lenguaje**
- **Modo IN-constante No** indica cómo se implementa:
 - **Puede o no realizar copia.** (*dependerá del compilador del lenguaje*)
- **La implementación DEBE garantizar** que:
 - el **parámetro real no sea modificado** (*requiere un control*)

Ejemplo: C/C++ usa `const` en declaración

```
void ActualizarMax( const int x, const int y )  
{if ( x > y ) Max= x ;  
  else Max= y ;}
```

Parámetros

Datos - Modo IN - Por valor constante:

Desventajas:

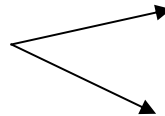
- **No permite modificar el parámetro**
- **Requiere incluir controles en la implementación.**

Ventajas:

- **Evita modificaciones accidentales**
- **Mayor seguridad, protección de datos**
- **Claridad en el uso del parámetro**

Parámetros

Diferentes formas de transmitir los parámetros

Datos - Modo OUT  **Por Resultado**
Por Resultado de funciones

El Parámetro Formal devuelve el resultado al Parámetro Real.

La conexión se da al final de la ejecución de la rutina.

Parámetros

• Datos - Modo OUT - Por Resultado:

- La **conexión** entre parámetros es **al finalizar**
- El **valor** del **parámetro formal** (rutina) se **copia al parámetro real al terminar de ejecutarse** la unidad llamada.
- El **parámetro formal** es una **variable local** del **entorno de la rutina**
- El **parámetro formal** es una **variable sin valor inicial** porque **no recibe nada**.
- **No** se debe **asumir que existe inicialización por defecto**. (*Dependerá de cada lenguaje*)
- El **parámetro no tiene** un **valor inicial definido**; la rutina **debe** **asignarlo antes de finalizar**

Parámetros

• Datos - Modo OUT - Por Resultado:

■ *Desventajas:*

- **Consume tiempo y espacio** porque **hace copia al final** (*pensar grandes volúmenes de datos*)
- El **parámetro formal no tiene valor inicial** (*puede generar errores*)
- **Requiere** que la **rutina asigne un valor antes de finalizar**
- **Puede generar problemas si no se inicializa correctamente**

■ *Ventaja:*

- **protege** los **datos** de la **unidad llamadora**, el **parámetro real no se modifica** durante la ejecución de la unidad llamada. (**No hay vínculo**)
- **Recibe datos que se copian al final**

Parámetros

Datos – Modos OUT– Por Resultado

Program main

var i:integer;

Procedure P(res a:integer)

var x,y: integer;

Begin

a=3

Inicializa a

x=a+1;

y=x+1;

a=y;

Asigna valor a devolver

end;

Begin

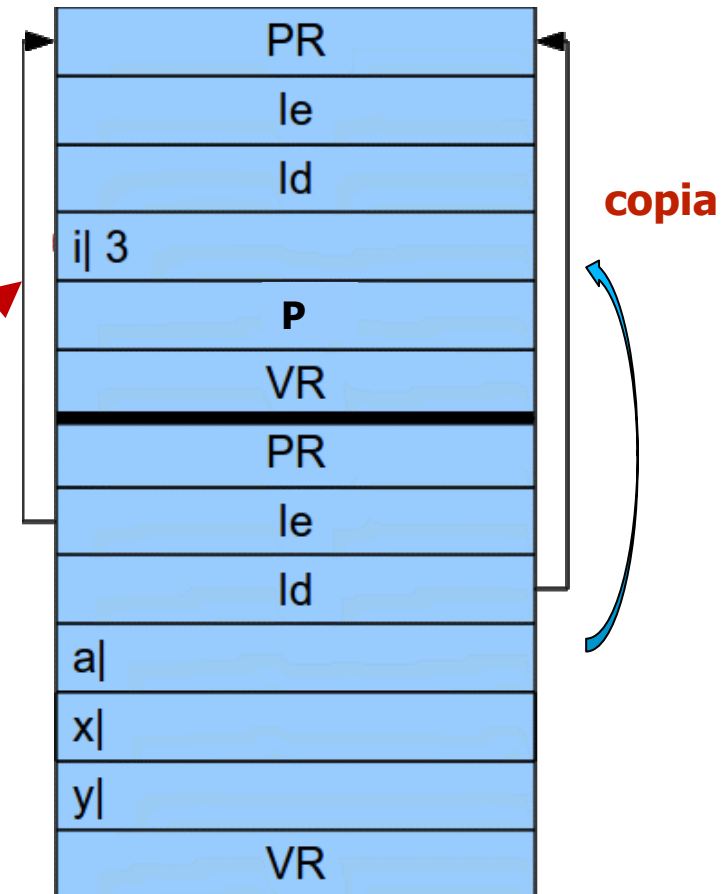
i=3;

P(i);

Print(i);

End.

Imprime 5



Antes de desalocar el registro de memoria se copia el valor del parámetro Formal en el registro que llamó al proc o fun.

Parámetros

RESULTADO FINAL

Datos – Modos OUT– Por Resultado

Program main

var i:integer;

Procedure P(res a:integer)

var x,y: integer;

Begin

a=3

Inicializa a

x=a+1;

y=x+1;

a=y;

Asigna valor a devolver

end;

Begin

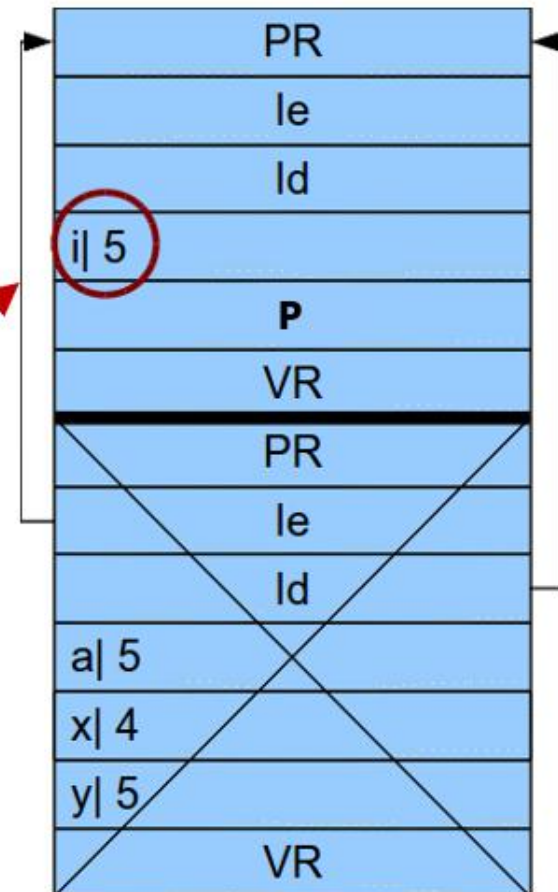
i=3;

P(i);

Print(i);

End.

Imprime 5



Antes de desalocar el registro de memoria se copia el valor del parámetro F. en el registro que llamó al proc o fun.

Parámetros

- **Datos - Modo OUT - Por Resultado de funciones:**
 - Es el **valor** que devuelve una función
 - El **resultado** se **reemplaza** directamente en la **expresión** donde se invocó la función
- **Formas de devolución según lenguajes:**
 - **Return**: devuelve valor o expresión (Ej.: Python, C)
 - **nombre de función**: el **nombre** de la **función** pasa a **ser** una **variable local** a la **función**, a la cual **se le asigna** el **valor** y que **será utilizada** para **pasar** el **valor** de **retorno** (Ej.: Pascal)

Parámetros

- **Datos - Modo OUT - Por Resultado de funciones:**
Ejemplos:

En C

```
int f1(int m){  
    m= 3;  
    return m + 5;  
}  
  
int x,y;  
x = f1(y); //x=8
```

En Pascal

```
function F1(m: integer): integer;  
begin  
    m= 3;  
    F1 := m + 5;  
end;  
  
var x: integer; y: integer  
begin  
    x := F1(y); //x=8  
end.
```

El valor retornado se reemplaza en la expresión de la invocación

Parámetros

- **Datos - Modo IN/OUT**
 - Por **Valor-Resultado**
 - Por **Referencia**
 - Por **Nombre**

Permiten **recibir un valor y devolver un resultado**
Conexión al inicio y al final

Parámetros

Datos - Modo IN/OUT - Por Valor/Resultado:

- La **rutina recibe** un **valor** y **devuelve** un **resultado**.
- Con la **invocación de la rutina**, se **copia** el **parámetro real** al **parámetro formal** y se **desvincula**.
- La **rutina trabaja** sobre la **copia local** del **parámetro formal** y el **parámetro real no se modifica**.
- El **parámetro formal** es una **variable local**. **No comparte memoria con el parámetro real**
- Al **finalizar** el **valor** del **parámetro** se **copia** al **parámetro real**.
- Hay **ligadura y conexión** entre **parámetro real** y el **formal** se da cuando **inicia la ejecución** de la rutina y cuando **termina**, **pero no hay vínculo durante la ejecución**. Hay copia al inicio y al final
- Tiene las desventajas y las ventajas del IN y del OUT.

Parámetros

Modos IN/OUT- Por Valor/Resultado

Program main

var i:integer;

Procedure P(in-out a:integer)

var x,y: integer;

a=4

x=a+1;

y=x+1;

a=y;

Begin

end;

Begin

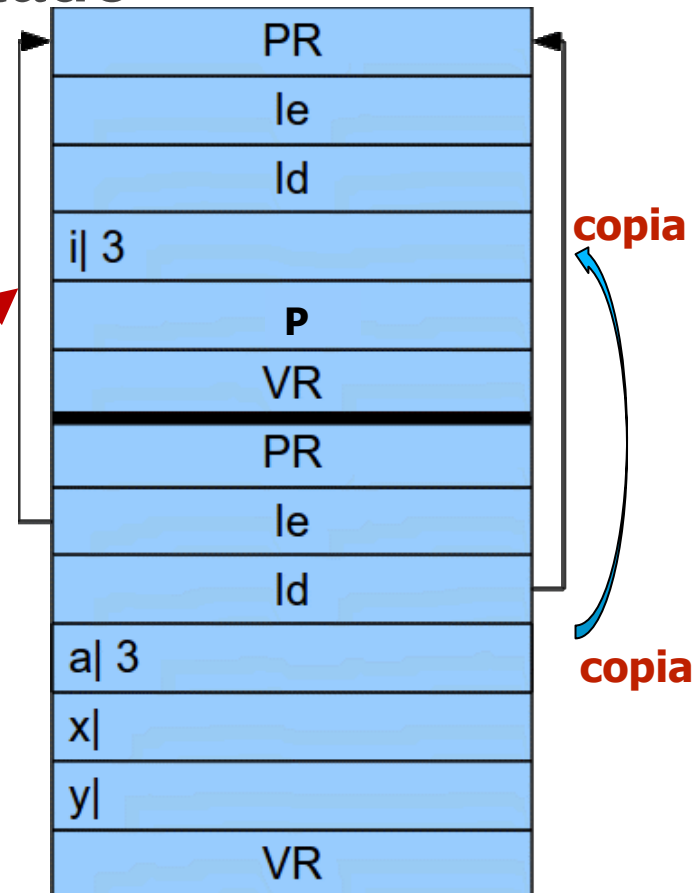
i=3;

P(i);

Print(i);

End.

Imprime 6



- ✓ Se copia el valor del PR al PF al alocar el registro de la rutina,
- ✓ Se trabaja sobre la copia local (parámetro formal)
- ✓ al finalizar la ejecución de la rutina se copia el PF al PR antes de desalocar.

Parámetros

Modos IN/OUT- Por Valor/Resultado

Program main

var i:integer;

Procedure P(in-out a:integer)

var x,y: integer;

Begin

a=4

x=a+1;

y=x+1;

a=y;

end;

Begin

i=3;

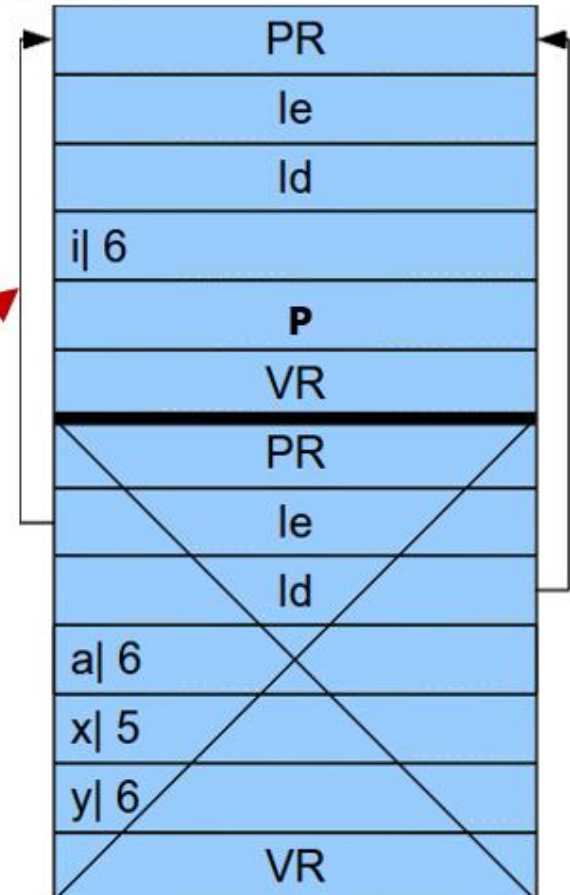
P(i);

Print(i);

End.

Imprime 6

RESULTADO FINAL



- ✓ Se copia el valor del PR al PF al alocar el registro de la rutina,
- ✓ Se trabaja sobre la copia local (parámetro formal)
- ✓ al finalizar la ejecución de la rutina se copia el PF al PR antes de desalocar.

Parámetros

Ejemplo con variable J local y M no local

Modos IN/OUT– Por Valor/Resultado

Procedure A ();

var m: integer;

Procedure B (valor-resultado j:
integer);

begin

j := j+5;

j:= j+ m;

write (j,m);

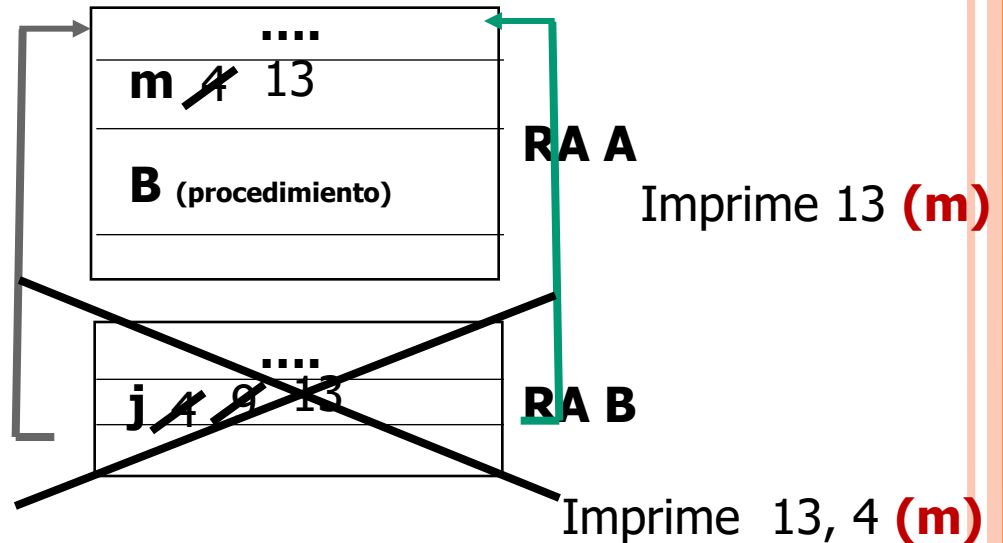
end;

begin

m:=4; B(m);

write (m);

end;



En el entorno de B:

- **j es "local"**
- **m es variable "no local" que pertenece al entorno de A**
- **Busca m por LE o LD**
- **computa**
- **m es modificada al final al recibir la copia (no en medio)**

El parámetro formal es una copia local que solo actualiza al real al finalizar.

Parámetros

Datos - Modos IN/OUT— Por Referencia:

- También llamada: **pasaje por referencia** o “por **variable**”
- **No se copia el valor, se copia la referencia a una dirección que es el */-valor* del Parámetro Real**
- La **conexión** es al **inicio y permanece hasta el final**
- El **PF** es una **variable local al entorno de la unidad llamada** que contiene la **dirección** del **PR** de la unidad llamadora,
- El **PF** apuntará entonces en un **ambiente no local**.
Se extiende el alcance de la rutina (*aliasing situation*)
- **Cada referencia al PF será a un ambiente no local,** entonces **cualquier cambio** que se realice en el **PF** dentro del cuerpo del subprograma **quedará registrado en el PR**.
El cambio será automático.
- **El PR queda compartido por la unidad llamadora y llamada. Será bidireccional.**

Parámetros

Datos - Modos IN/OUT– Por Referencia:

Desventajas:

- Puede **modificarse el PR inadvertidamente**. Es el **peor problema: pérdida de control y errores**
- Generar **efectos colaterales** al **dificultar la comprensión del programa**
- El **acceso indirecto al dato** puede implicar **menor eficiencia** en algunos casos
- Puede producirse un **aliasing** con **múltiples referencias** (**distintos nombres apuntando a la misma dirección de memoria** con un **comportamiento difícil de predecir**)

Impacto: afecta la **legibilidad, confiabilidad, y depuración de errores**

Parámetros

Datos - Modos IN/OUT– Por Referencia:

Ventajas:

- **No hay copias de datos**, esto lo hace **eficiente** en **uso** del **espacio y tiempo requerido** (*grandes volúmenes de datos*)
- **Adecuado** para **estructuras grandes e intercambio de datos**
- **Es fácil de implementar en muchas arquitecturas** (*la indirección es un mecanismo de bajo costo de implementación*)

Parámetros

Ejemplo de Aliasing básico

Datos – Modos IN/OUT– Por Referencia

Program main;

var i:integer;

Procedure P(var a:integer);

var x,y: integer;

a:=4;

x:=a+1;

y:=x+1;

a:=y;

Begin

end;

Begin

i:=3;

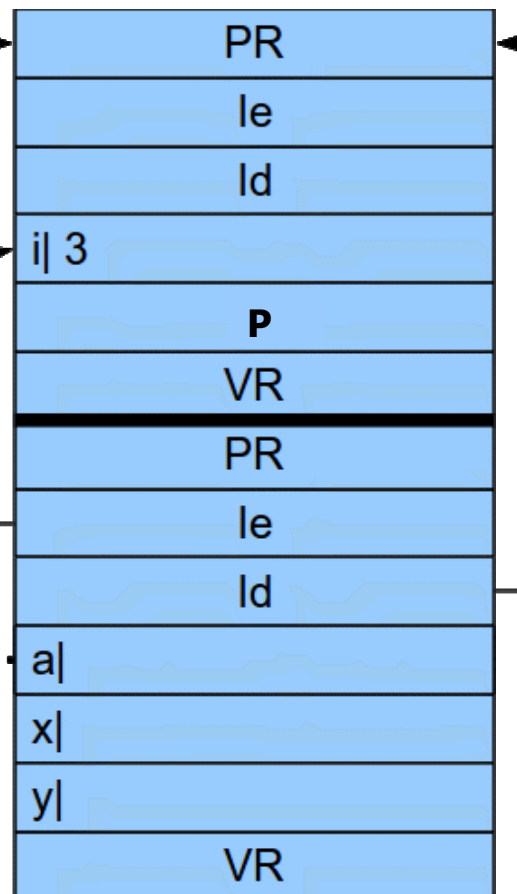
P(i);

Print(i);

End.

Imprime 6

L-valor de a



- ✓ Se trabaja directamente sobre el parámetro real a través del formal. Cambios inmediatos
- ✓ Por Referencia también es conocido por Variable (var)

Parámetros

Ejemplo de Aliasing básico

Datos – Modos IN/OUT– Por Referencia

Program main

var i:integer;

Procedura P(var a:integer)

var x,y: integer;

Begin

a=4

x=a+1;

y=x+1;

a=y;

end;

Begin

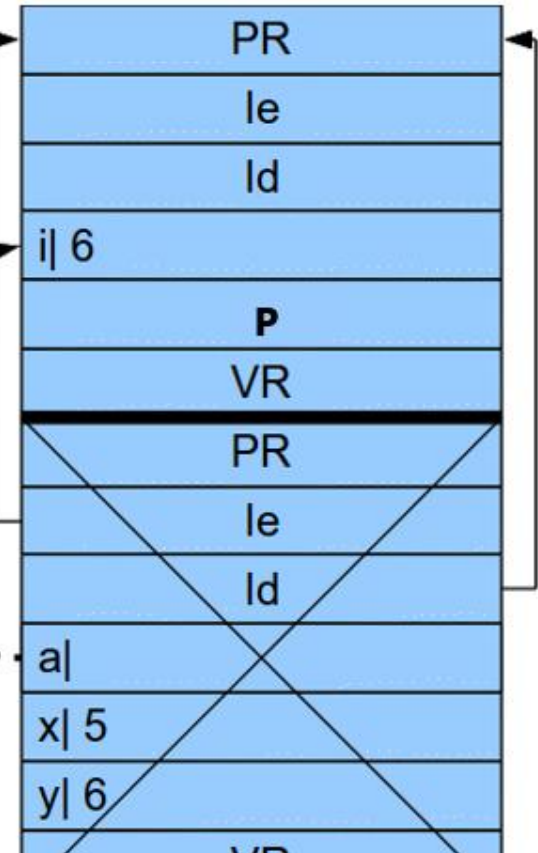
i=3;

P(i);

Print(i);

End.

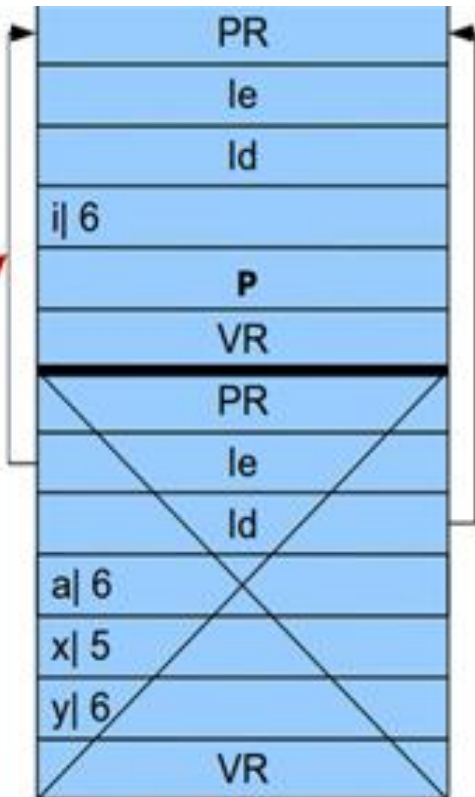
Imprime 6



- ✓ Se trabaja directamente sobre el parámetro real a través del formal
- ✓ Por Referencia también es conocido por Variable (var)

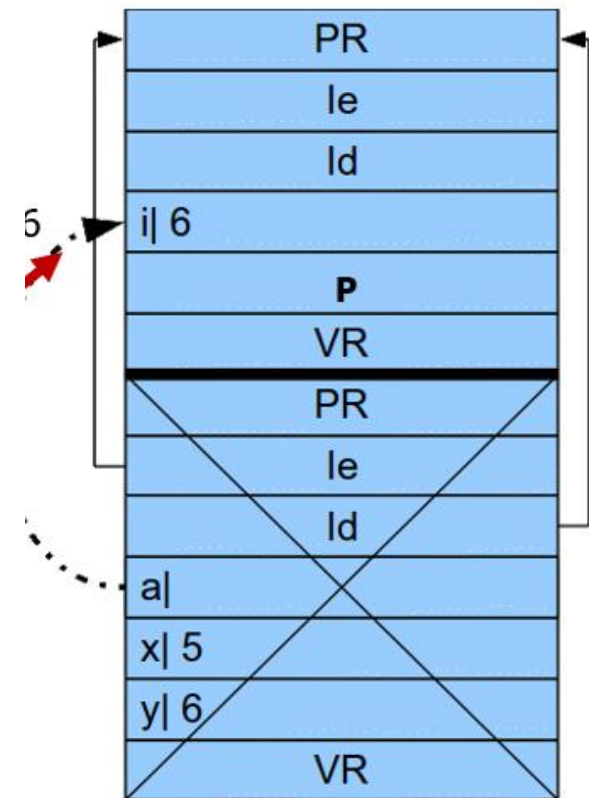
Parámetros – Comparación

Por Valor/Resultado versus Por Referencia



Diferencia fundamental:

- **Valor-resultado**
 - **copia** al **inicio** y al **final**
 - **sin vínculo** durante la **ejecución**
- **Referencia**
 - **acceso directo** al **parámetro real**
 - **cambios inmediatos**



Comparación de ambas modalidades:

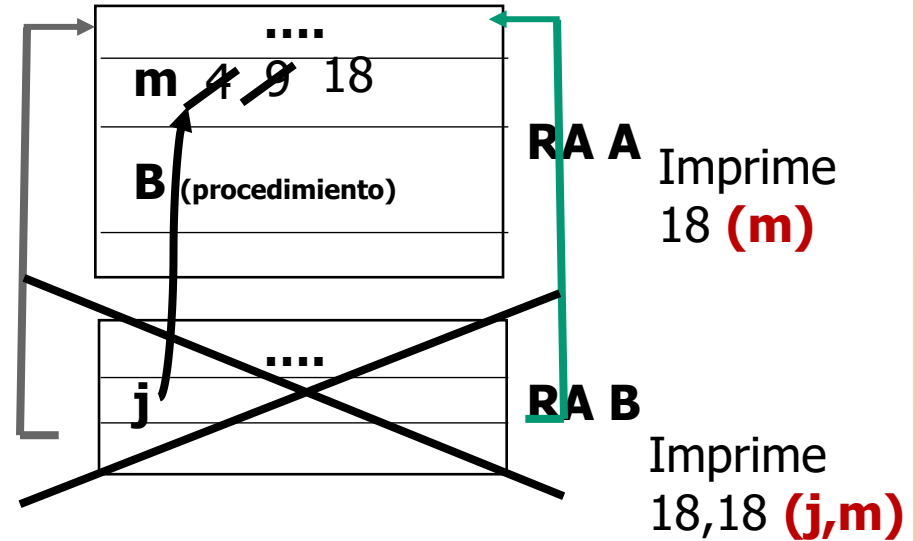
- ✓ Se obtiene mismo resultado
- ✓ Son semánticamente distintos

Parámetros

Datos – Modos IN/OUT– Por Referencia

```
Procedure A ();  
var m: integer;  
Procedure B (var j: integer);  
begin  
    j := j + 5;  
    j := j + m;  
    write (j, m);  
end;  
begin  
    m:=4;  
    B(m);  
    write (m);  
end;
```

Ejemplo de Aliasing más acceso a variables no locales



Caso:

- **j** parámetro formal local (de B)
- **m** variable no local (de A)
- **m (de j+m) la busco mediante el link estático (LE) o el link dinámico (LD) (según implementación)**
- **m es modificada siempre**

- ✓ **j apunta a m por referencia,**
- ✓ **ambos representan misma dirección de memoria,**
- ✓ **Impacto: dificulta la comprensión puede generar errores, afecta la confiabilidad del programa**

Parámetros

Datos – Modos IN/OUT– Por Referencia

¿Por qué este código puede ser engañoso?

- Porque **parece operar** con **dos variables**, pero en realidad **es una sola**

¿Por qué no siempre conviene usar pasaje por referencia?

- Puede **generar efectos no evidentes**
- Una **variable** puede tener **múltiples nombres (aliasing)**
- **Dificulta** la **comprensión** del programa
- Puede **generar errores** y afectar la **confiabilidad**

¿Cuándo conviene utilizarlo?

- Cuando se necesita **modificar** el **valor original**
- Para **evitar copias** (mayor **eficiencia**)
- Para trabajar con **estructuras grandes**

¿Quién define su uso?

- **Depende** del **lenguaje** (qué mecanismos **permite**)
- Y del **programador** (si puede **elegir** cómo **pasar parámetros**)



Parámetros

Datos - Modo IN/OUT - Por Nombre:

- Introducido por **ALGOL60**
 - **No se utiliza actualmente en lenguajes imperativos.**
 - **Método conceptualmente importante.**
- La **idea** general es que todo **parámetro formal** se reemplaza por su **parámetro real**. (*Regla de la Copia*)
- **Problema:** no se puede hacer directo porque **pueden existir variables con el mismo nombre en ambos entornos** y el **resultado** puede **cambiar** si **se evalúa** en **otro entorno**.
- **Entonces:** el **parámetro** **forma** debe **evaluarse** en el **entorno del llamador**
- **Solución:** **sustituir el Parámetro Formal por el nombre del Parámetro Real** junto con su propio entorno de evaluación

**El parámetro no es un valor, es una expresión
que se evalúa en el entorno del llamador**

Parámetros

Datos - Modo IN/OUT - Por Nombre:

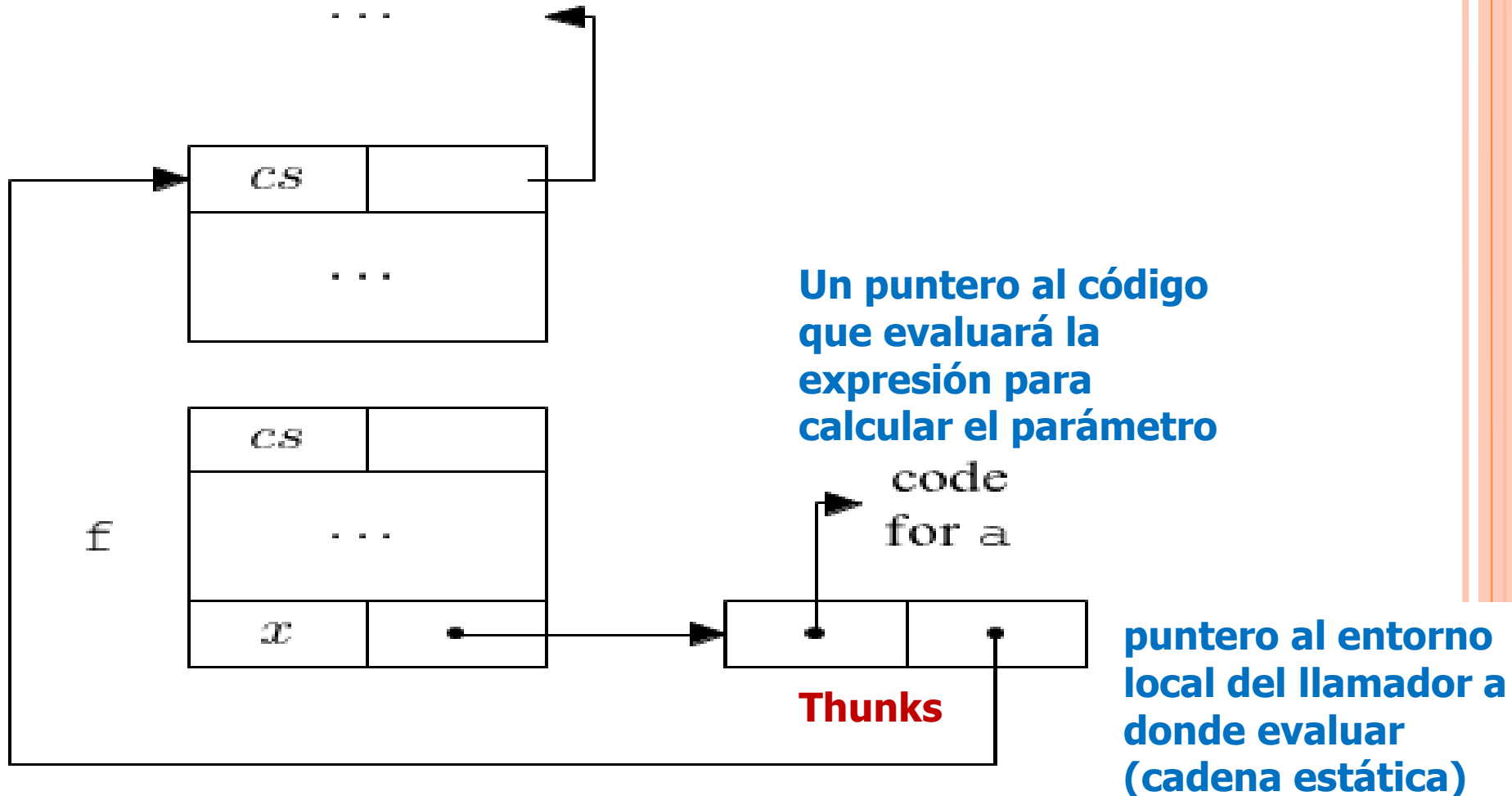
- **Parámetro formal se reemplaza por (*exp*, *ptr*):**
 - ✓ **una expresión textual** del parámetro real +
 - ✓ **un puntero al entorno** del parámetro real
- **Se implementa con una estructura que tiene:**
 - **código** para **evaluar** la **expresión**
 - **referencia** al **entorno** del **llamador**
- **La ligadura de:**
 - ✓ **Parámetros** (entre PF y PR) en **invocación**
 - ✓ **Valor: diferida** hasta que **se utiliza el parámetro** (***la dirección se resuelve en ejecución***).
- El **objetivo** es otorgar **evaluación de valor diferida**.

Semánticamente distinto a "Pasaje por Referencia", no se pasa un valor, se pasa una forma de calcularlo.

Parámetros

Datos - Modos IN/OUT– Por Nombre (continuación):

Implementación Thunks en llamada por nombre



Parámetros

Datos - Modos IN/OUT– Por Nombre (continuación):

- **Thunks** es:
 - una **unidad pequeña de código (función) que representa a una expresión** (*encapsulamiento*)
 - ***Y pospone su evaluación hasta que sea necesario*** (*evaluación perezosa o diferida*)
- Para implementar el pasaje por Nombre:
 - El **parámetro formal se reemplaza** en el cuerpo de la unidad llamada **por una invocación al thunks**,
 - **cada uso del parámetro ejecuta ese thunk**
 - Durante la **ejecución** **evaluará ese parámetro real en el entorno del llamador.**

Parámetros

Caso: índice de un vector como parámetro por nombre

Datos – Modos IN/OUT– Por Nombre

Program main

var i:integer;

Procedure P(nombre a:integer)

var vec[1..3] of integer;

Begin

vec[1]=0;

a=a-1;

vec[i]=a;

vec[a+1]=1;

end;

Begin

i=3;

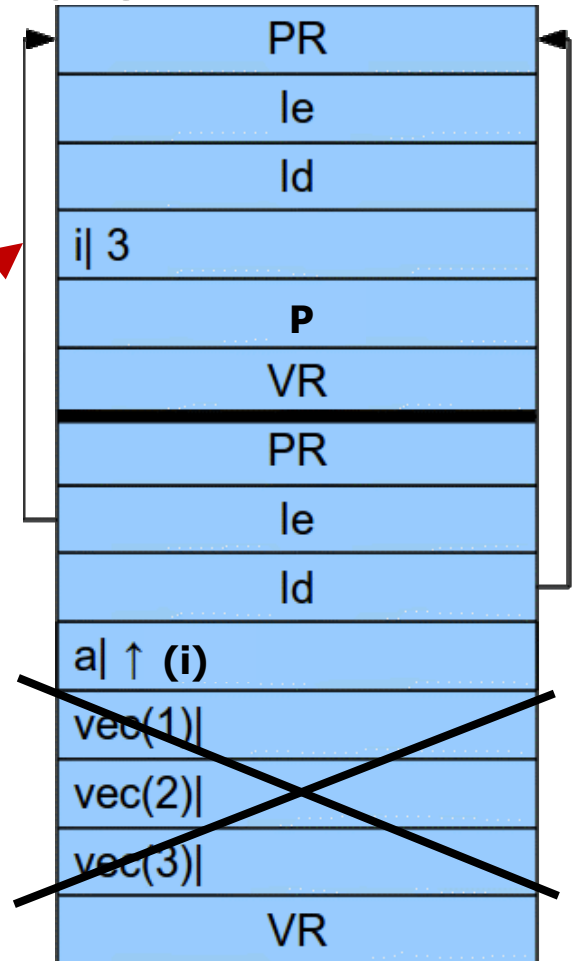
P(i);

Print(i);

End.

Imprime
2

a = a - 1; es
i = i - 1;
etc



El PF es sustituido textualmente apuntando al PR
¿que significa a?

Una expresión que se evalúa cada vez

Parámetros

Caso: índice de un vector como parámetro por nombre

Datos – Modos IN/OUT– Por Nombre

RESULTADO FINAL

Program main

var i:integer;

Procedure P(nombre a:integer)

var vec[1..3] of integer;

Begin

vec[1]=0;

a=a-1;

vec[i]=a;

vec[a+1]=1;

end;

Begin

i=3;

P(i);

Print(i);

End.

Imprime
2

PR
le
ld
i 2
P
VR
PR
le
ld
a ↑ (i)
vec(1) 0
vec(2) 2
vec(3) 1
VR

**El PF es sustituido textualmente apuntando al PR
¿que significa a?**

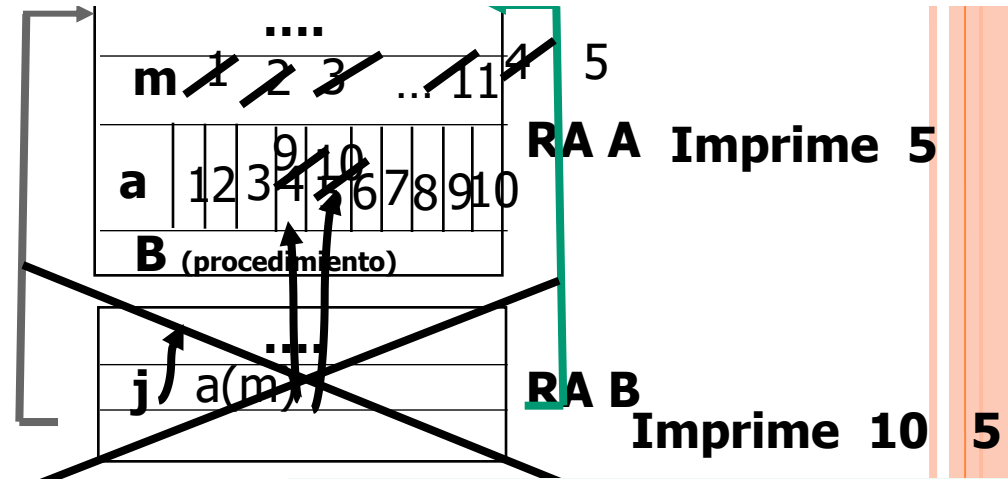
Una expresión que se evalúa cada vez

Parámetros

Caso: un vector como parámetro por nombre

Datos – Modos IN/OUT–
Por Nombre

```
Procedure A ();  
  var m:integer;  
  a: array[1..10] of integer;  
Procedure B (nombre j:integer);  
  begin  
    j:=j+5;  
    m:=m+1;  
    j:= j+ m;  write (j,m);  
  end;  
begin  
  for m=1 to 10 begin  a[m]=m;  end;  
  m:=4;  B(a[m]);  
  write (m);  
end;
```



- ✓ **m** es local a Proc A
- ✓ Se recalcula cada vez la dirección **a[m]** durante la ejecución
- ✓ el parámetro puede referir a distintas posiciones de memoria en distintos momentos
- ✓ Preguntarse que significa **j=a(m)** en el registro de activación del "llamante"

Parámetros

Datos - Modos IN/OUT– Por Nombre:

■ Desventajas:

- **Extiende el alcance del procedimiento**
 - **accede al entorno del llamador** y puede **generar errores**
- **Evaluación diferida**
 - el **parámetro** se **evalúa** en **cada uso**
- **Ineficiencia**
 - **evaluación repetida** del **parámetro real**, puede ser **costoso** (ej: dentro de loops)
- **Puede existir aliasing**
 - **múltiples accesos** a una **misma variable**
- **Complejidad**
 - **difícil de implementar**, difícil de **entender** y puede generar **soluciones confusas**

Parámetros

Datos - Modos IN/OUT– Por Nombre (cont.):

■ Ventajas:

- **Gran flexibilidad**
permite **pasar expresiones**, no solo variables
- **Evaluación diferida**
el **valor** se **calcula** solo **cuando** se **necesita**
- **Evitar cálculos innecesarios**
útil si el **parámetro** no siempre **se usa**
- **Permite comportamientos dinámicos**
el **resultado** puede **cambiar según** el **entorno**
- **Importancia conceptual**
base de técnicas modernas como **evaluación perezosa**
permite **pasar** no solo datos, sino **formas de calcular datos**

Parámetros

■ Pasaje de parámetros en algunos lenguajes:

■ C:

- **Por valor**, (si se necesita ***por referencia se usan punteros***).
- permite pasaje **por valor constante**, agregándole ***const***

■ Pascal:

- **Por valor** (por defecto)
- **Por referencia** (opcional: **var**)

■ C++:

- Similar a C
- Más pasaje **por referencia**

■ Java:

- Sólo **copia de valor**. Pero como las variables de tipos **no primitivos** son **todas referencias** a variables anónimas en el HEAP, el paso por valor de una de estas variables constituye en realidad un **paso por referencia** de la variable.
- La **estructura de datos primitiva** es un tipo de estructura de datos que **almacena datos de un solo tipo**. Ejemplos ***son los enteros, los caracteres y los flotantes, etc.***
- La estructura de datos **no primitiva** es un tipo de estructura de datos que **puede almacenar datos de más de un tipo**. Ejemplos ***son los arrays, TAD, listas, etc.***

Parámetros

- **Pasaje de parámetros en algunos lenguajes (continuación):**
 - **PHP:**
 - **Por valor**, (predeterminado).
 - **Por referencia** (&)
 - **RUBY:**
 - **Por valor.** Pero al igual que Python si se pasa es un objeto “**mutable**” (objeto que puede ser modificado), no se hace una copia, sino que se trabaja sobre él.
 - **Python:**
 - Envía objetos que pueden ser “**inmutables**” o “**mutables**” (objeto que pueden ser o no modificados). Si es **inmutable** actuará como **por valor** y, si es **mutable**, ejemplo: listas, no se hace una copia, sino que **se trabaja sobre él**.

Parámetros

- **Pasaje de parámetros en algunos lenguajes (continuación):**
 - **ADA:** usa varios
 - **Por copia modo IN** (por defecto)
 - **Por resultado modo OUT**
 - **IN-OUT.**
 - Para los tipos de datos **primitivos** indica que es por **valor-resultado**
 - Para los tipos **no primitivos**, y datos compuestos (arreglos, registros) se hace por **referencia**
 - Particularidad de ADA:
 - **En las funciones** solo se permite el paso por **copia de valor**, lo cual **evita** parcialmente la posibilidad de **efectos colaterales**.

Los tipos primitivos de datos proveídos por Ada son seis

1. **Integer** : Tipo que abarca los enteros positivos y negativos Natural : Este tipo de dato contiene números positivos más el cero.
2. **Positive**: Solamente permite números positivos.
3. **Float**: Tipo que almacena cualquier número real
4. **Char** : Guarda un carácter.
5. **String**: Almacena un número definido de caracteres .

Parámetros

- ✓ Hasta ahora pasábamos:
 - **datos como parámetros**
- ✓ También se pueden pasar:
 - **subprogramas como parámetros**

Podemos pasar comportamiento, no solo datos

Parámetros

3) Subprogramas como Parámetros:

1. Rutinas que reciben funciones como parámetros
 - Bastante común en la mayoría de los lenguajes
2. Rutinas que devuelven funciones como resultado
 - Fundamental en lenguajes funcionales.
 - Poco frecuente en lenguajes imperativos

Parámetros

3) Subprogramas como Parámetro - Usos

Se utilizan **cuando** se **desea ejecutar** distintas **acciones** de **forma flexible**, **pasando** el **nombre de subprograma** como **parámetro**

Ejemplos de aplicación:

- **Procesamiento de arreglos:**
se pasa el subprograma que define la operación (*ej: ordenamiento, búsqueda, etc.*)
- **Manejo de eventos o errores:**
se pasa una rutina que define el *comportamiento* (*ej. manejador de excepciones*)
- **Funciones matemáticas:**
se pasa la función matemática que se desea aplicar

Parámetros

3) Subprogramas como Parámetro - Problemas

Caso: una función $f()$ pasa como parámetro otra función $f(g())$

Preguntas:

- **¿qué entorno se utiliza** cuando el subprograma **accede a variables no locales?**
- **¿hay variables** con el **mismo nombre** que están **en diferentes entornos?**
- **¿hay subprogramas con funciones anidados?**
- **¿hay múltiples entornos activos?**

Resolver:

- **¿Cuál** es el **entorno de referencia?**
- **¿Cómo** se realiza el **chequeo de tipos?**
- **¿Cómo** se **manejan subprogramas anidados?**

Ahora el problema es en qué entorno se ejecuta el código que paso

Parámetros

- **Ambiente de referencia para subprogramas pasados como parámetros y acceso a variables no locales**
- **Debe determinarse** cuál es el **ambiente de referencia no local correcto** para un **subprograma pasado** como **parámetro** y luego **invocado**
- **Cuando** este **subprograma** se **ejecuta** pueden **surgir problemas** al **resolver referencias** a **variables no locales**
- **En general:**
 - **si** el **subprograma** **accede** a **variables** que **no le pertenecen**, surge la **pregunta** de **dónde deben buscarse**, ya que puede **ejecutarse** en un **entorno distinto** al **de su definición**
- **El comportamiento depende del lenguaje:**
 - **Cadena estática:** entorno donde fue definido
 - **Cadena dinámica:** entorno desde donde fue invocado

Parámetros

- **Ambiente de referencia para las referencias no locales dentro del cuerpo del subprograma pasado como parámetro.**
- **Que pasa si se dan estas condiciones:**
 - Se **pasan funciones como parámetros**
 - Esas **funciones usan variables no locales**
(variables libres: se usa dentro de un subprograma, pero no está declarada en él)
 - **Existen subprogramas anidados**
- **Problema**
 - ¿a qué entorno pertenecen esas variables?
 - ¿dónde se deben buscar esas referencias?

Veremos reglas de ligadura y alcance

Parámetros

- Ambiente de referencia para las referencias no locales dentro del cuerpo del subprograma pasado como parámetro.

Reglas de ligadura y alcance:

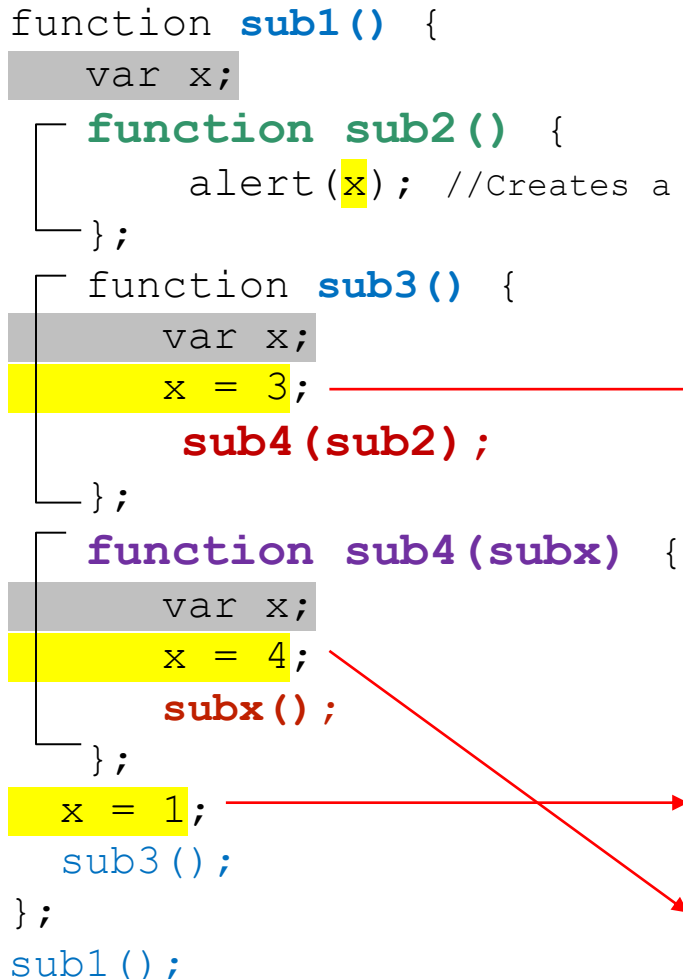
- Ligadura shallow (*superficial*)
- Ligadura deep (*profunda*)
- Ligadura ad-hoc

Estas **reglas determinan** en qué **entorno** se **evalúa** el **subprograma** y sus **variables no locales** cuando se ejecuta

Parámetros – Ejemplo en JS

La ejecución de sub2 cuando se llama que valor de X usará? 4, 1, 3?

```
function sub1 () {  
  var x;  
  function sub2 () {  
    alert(x); //Creates a dialog box with the value of x  
  };  
  function sub3 () {  
    var x;  
    x = 3;  
    sub4(sub2);  
  };  
  function sub4(subx) {  
    var x;  
    x = 4;  
    subx();  
  };  
  x = 1;  
  sub3();  
};  
sub1();
```



sub4(sub2); //paso a sub2
subx(); //ejecutará sub2
Subx() parámetro (PF)
Sub2() argumento (PR)

Ad hoc: Busca en el entorno donde se pasa la función como argumento, es decir, en el subprograma desde donde se realiza el pasaje (sub3). Usa la x local en sub3, y la salida será **x=3 (AMBIENTE B)**
menos común en la práctica.

Deep/Profunda: Busca en el entorno donde se define la función pasada como argumento, es decir, en el subprograma donde se declaró (sub1). Usa la x local en sub1, y la salida será **x=1 (AMBIENTE A)**
Es el estándar en lenguajes con alcance estático.

Shallow/Superficial: Busca la variable en el entorno donde ejecuta/llama la función pasada como argumento, es decir, en el subprograma que la invoca (sub4). Usa la x local en sub4, la salida será **x=4 (AMBIENTE C)**
Es común en lenguajes con alcance dinámico *

Bibliografía

- **GHEZZI C. - JAZAYERI M.: Programming language concepts. John Wiley and Sons. (1998) 3er. Ed**
- **SEBESTA: Concepts of Programming languages. (2019) 12 Ed.**
- **David A. Watt, William Findlay: Programming language design concepts (2004)**
- **Maurizio Gabbrielli and Simone Martini: Programming Languages: Principles and Paradigms (2010)**